

Range Through Chart and Diversity Curves in PaleobiologyDB (Coded in Julia)

Jeet Sukumaran and Maritza Sardina

Table of contents

1	Part 1: Indicating the Ranges of Each Organism through Geological Time	2
1.1	Setting up the Julia environment	2
1.2	Retrieving data for our fossils of interest, using Edrioasteroidea as an example . .	3
1.3	Evaluating the range of all fossil occurrences for our fossils of interest	5
1.4	Simplified workflow for evaluating the range of fossil occurrences of interest . .	7
2	Part 2: Constructing a Diversity Curve	10
2.1	Preparation by counting diversity per geographical interval	10
2.2	Creating DataFrames for each fossil	14
2.3	Plotting a diversity curve	14
2.4	Why are we doing all of this?	17
3	Part 3: Answering Activity Questions	17
4	References (for Original Activity)	18

The original tutorial for “Range Through Chart and Diversity Curves Lab” was made possible by Katherine Bulinski from Bellarmine University, and was originally published on June 4th, 2009. This tutorial incorporates the original tutorial using a mix of both Julia code and written questions, as well as use of the PaleobiologyDB’s Navigator tool and the Sepkoski curve as a reference. This tutorial can be accessed at <https://serc.carleton.edu/NAGTWorkshops/paleo/activities/33101.html>.

Major events have occurred throughout geological history, “[including] the Cambrian explosion, the Ordovician Radiation, mass extinction events [”Big Five”], and more” (Bulinski, 2009). Many of these events can be analyzed and reconstructed in ways that measure biodiversity, or specifically the changes in worldwide diversity over geological time. For instance, the Sepkoski curve measures the biodiversity of numerous datasets and fossil records through time, which is done by counting the number of families in marine ecosystems throughout history and the

changes that occur in their biodiversity. For more information, this curve can be accessed at <https://silurian-reef.fieldmuseum.org/narrative/474>, and it should be used in conjunction with your data in answering the questions at the bottom.

The main goal of this tutorial is to establish familiarity with twenty groups of organisms that have existed throughout geological time, knowledge about the paleontologic events throughout history, and understanding about the basics, commands, and functions of the Julia programming interface. Furthermore, you should interpret the results of your data and visualizations to the best of your ability when answering the questions indicated throughout the tutorial.

1 Part 1: Indicating the Ranges of Each Organism through Geological Time

1.1 Setting up the Julia environment

To start this tutorial, you'll need to set up your Julia environment with all necessary packages. Using Julia's built-in package manager (Pkg), you can install all required packages, then load them into your environment as follows:

```
using Pkg

Pkg.add(["PaleobiologyDB", "DataFrames", "CairoMakie", "GeoMakie"])

using PaleobiologyDB, DataFrames, CairoMakie, GeoMakie
```

NOTE: To (re)install the “PaleobiologyDB.jl” package, along with its additional modules (including Taxonomy and PhyloPicMakie), you'll need to load Pkg into the Julia environment, then install the package(s) through URL rather than by name. This can be done using `Pkg.add(url = "website name")`, with the actual URL being inside the links. In this case, we will be installing the “PaleobiologyDB.jl” package through the Jeet Sukumaran GitHub website (<https://github.com/jeetsukumaran/PaleobiologyDB.jl>). As such:

```
using Pkg

Pkg.add(url = "https://github.com/jeetsukumaran/PhyloPicMakie.jl")
Pkg.add(url = "https://github.com/jeetsukumaran/PaleobiologyDB.jl")
```

1.2 Retriving data for our fossils of interest, using Edrioasteroidea as an example

For this part, there are twenty different fossils (classes and orders) indicated in the original tutorial, from Edrioasteroidea (class) all the way to Insecta (class). For the sake of this tutorial, we'll be walking through the workflow for the first fossil, Edrioasteroidea, and then the workflow can be used for the remaining nineteen fossils. We will first name Edrioasteroidea as a focal taxon, then to use `pbdb_taxa` to retrieve more information about the taxonomic ranks of Edrioasteroidea:

```
focal_name = "Edrioasteroidea"

taxon_names_df = pbdb_taxa(
  name = "Edrioasteroidea",
  rel = "all_children",
  show = "full",
  vocab = "pbdb",
)

taxon_names_df
```

The function `filter` will be used to return the vector of taxonomic numbers of Edrioasteroidea to those solely including families, which is shown as follows:

```
filter(row -> row.accepted_rank == "family", taxon_names_df)
```

We will use `pbdb_occurrences` to retrieve the data for fossil occurrences of Edrioasteroidea with a unique variable name. Afterwards, we will inspect and analyze the data using a variety of different functions, including `describe`, `show`, and `filter`. The `show` function will be used to list everything from the raw DataFrame, including all rows and columns. The end result will be an almost spatial-like textual diagram of every single row and column name of the DataFrame, which may look messy. `describe` will then be used to summarize the names of the DataFrame, and `filter` will be used to filter out our data. In this case, we want to filter out the data to solely include genera and species. The code for this workflow is shown as follows:

```
occs_edri = pbdb_occurrences(
  base_name = "Edrioasteroidea",
  show = "full",
  vocab = "pbdb"
)

show(occs_edri, allrows = true, allcols = true)
```

```
describe(occs_edri)

filter(row -> row.accepted_rank == "genus", occs_edri)

filter(row -> row.accepted_rank == "species", occs_edri)
```

For the sake of this tutorial, we want to filter out our fossil occurrences to include only genera, which is our taxonomic rank of interest. `!ismissing` and `!isempty` are both used to filter out any missing or empty values in our DataFrame. The simplified version of the ‘filter’ function workflow will thus be:

```
filter(row -> row.accepted_rank == "genus", occs_edri)
occs_edri_genera = filter(row -> !ismissing(row.genus) && !isempty(row.genus), occs_edri)
```

To print out just our variable by itself, we can simply type the name in our terminal, as such:

```
occs_edri
```

From there on out, we will conduct more analysis of our data set. Some of these examples include naming a function, counting strings through our named variable, printing out the names of certain columns and rows needed in our data (e.g., `x = :genus`, with a “:” before the word “genus”), and much more.

```
occs_edri[1,1]

occs_edri[1, [:phylum, :class, :order, :genus, :accepted_name]]

x = :genus

occs_edri[1, [x]]
```

Here, we are assigning variable names to certain columns in our DataFrames. Specifically, the `[]` brackets are used to index our variables/elements embedded in our DataFrame when functioning as an operator (e.g., `[1, 1]` is used to concentrate on the entire DataFrame). On the other hand, we can assign variables to columns, such as `:genus` (with the colon for getting across our syntax), then use these variables within our square brackets to easily access our assigned column syntax.

NOTE: These are examples of exercises that you can try out before moving on to the next section. For more exercises and practices, you can check out the following resources, tutorials, and additional courses below:

- [Julia Tutorials and Resources](#) (Official Julia tutorials page)
- [Range through time diversity analysis](#) (Sections 1.10, 2.11, and 3.10; Section 3.11 for split-apply-combine method golf practice)
- [Julia Programming Tutorial for Beginners](#) (from DataCamp)
- [JuliaHub Tutorials Page](#) (List of Community Resources for Julia)

1.3 Evaluating the range of all fossil occurrences for our fossils of interest

To evaluate the exact range of Edrioasteroidea fossil occurrences, we will use `pbdb_occurrences` once more to create a new range-through chart that evaluates the range and diversity of Edrioasteroidea fossils over time. Firstly, we would want to retrieve our raw data by setting up our environment with all necessary packages and setting up auto-caching. This specifically allows us to retrieve our data and to save it in our local REPL. As such:

```
using PaleobiologyDB, PaleobiologyDB.Taxonomy, DataFrames
using Makie, CairoMakie, GeoMakie, ColorSchemes

PaleobiologyDB.set_autocaching!(true);

occs_edri_raw = pbdb_occurrences(
    base_name = "Edrioasteroidea",
    show = "full",
    vocab = "pbdb"
)
```

Then, we would want to clean up our data using `dropmissing` (via `DataFrames`) to drop any missing values and `drop_unresolved_taxa` (through the `Taxonomy` module of `PaleobiologyDB.jl`) to drop all unresolved taxa in our `DataFrame`, specifically for our genera. To ensure that our data is clean, we will use this code:

```
occs_edri = dropmissing(occs_edri_raw, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])

occs_edri = drop_unresolved_taxa(occs_edri, :genus)
```

Next, we will need to calculate a midpoint age (Ma) for each Edrioasteroidea fossil occurrence, which we can do by creating a new code that calculates the midpoint age by dividing a sum of

maximums and minimums by two, via arithmetic operations. Then, we will sort out our data using `sort` to include a new row for our calculated midpoint ages. This code is as follows:

```
occs_edri.midpoint_age = (occs_edri.max_ma .+ occs_edri.min_ma) ./ 2

occs_edri = sort(occs_edri, [:genus, :midpoint_age])
```

To calculate our range through values, we will be using the split-apply-combine method. This method is a type of coding workflow that splits a bigger DataFrame (consisting of all fossil occurrences for our genera/species) into per-genus specific tables consisting of first appearance and last appearance data. This is done by applying a summary function to each sub-table and combining them to form a new DataFrame, as follows:

```
appearance_occs_edri = combine(
    groupby(occs_edri, :genus, sort = true),
    :midpoint_age => maximum => :first_appearance_datum,
    :midpoint_age => minimum => :last_appearance_datum
)
```

Before we can plot our diversity curve for Edrioasteroidea (and our other fossils), we will need to again sort our data by the data of first appearance for each occurrence (`:first_appearance_datum`) in a reverse order. This allows us to chart our data from oldest appearances to the most recent appearances.

```
appearance_occs_edri = sort(appearance_occs_edri, :first_appearance_datum, rev=true)
```

We will be manually creating DataFrames for both Edrioasteroidea and the other nineteen fossils of interest to ensure that everything is properly graphed on each range through chart, and subsequently, for the overall diversity curve graph. Ensure that you are using different variable names for each of the fossils. As such:

```
using DataFrames

range_through_chart_edri = DataFrame(
    genus = appearance_occs_edri.genus,
    first_appearance_datum = appearance_occs_edri.first_appearance_datum,
    last_appearance_datum = appearance_occs_edri.last_appearance_datum
)
```

To create a range-through chart for Edrioasteroidea, we will use both “Makie” and “CairoMakie” to create a figure and a single axis, as well as a loop function to plot the range of each Edrioasteroidea genus through index number `i` and components `x1` and `x2` for both first and last appearance data. The code for creating the chart is as follows:

```

using Makie, CairoMakie, ColorSchemes

fig = Figure(size = (800, 700))
ax = Axis(
    fig[1,1],
    xlabel = "Midpoint age (Ma) of Edrioasteroidea genera",
    ylabel = "Genera",
    yticks = (1:nrow(appearance_occs_edri), appearance_occs_edri.genus),
    yticklabelsize = 7,
    title = "Ranges and diversity of Edrioasteroidea genera",
    xreversed = true
)

for i in 1:nrow(appearance_occs_edri)
    x1 = appearance_occs_edri.first_appearance_datum[i]
    x2 = appearance_occs_edri.last_appearance_datum[i]
    lines!(ax, [x1, x2], [i, i], colorscale = :auto)
end

fig

```

1.4 Simplified workflow for evaluating the range of fossil occurrences of interest

For the remainder of the nineteen fossils, you can use the workflow established in the previous section to evaluate the range of all fossil occurrences for each class and order listed in the original tutorial. The taxon names for each of the fossils are as follows:

- **Edrioasteroidea** (class)
 - Range: Cambrian - Permian
- **Articulata** (class)
 - Range: Cambrian - Recent
- **Ammonoidea** (subclass)
 - Range: Silurian - Cretaceous
- **Rugosa** (order)
 - Range: Ordovician - Permian
- **Tabulata** (order)

- Range: Ordovician - Permian
- **Blastoidea** (class)
 - Range: Ordovician - Permian
- **Rostroconchia** (class)
 - Range: Cambrian - Permian
- **Gastropoda** (class)
 - Range: Cambrian - Recent
- **Stenolaemata** (class)
 - Range: Ordovician - Recent
- **Trilobita** (class)
 - Range: Cambrian - Permian
- **Scleractinia** (order)
 - Range: Triassic - Recent
- **Crinoidea** (class)
 - Range: Ordovician - Recent
- **Echinoidea** (class)
 - Range: Ordovician - Recent
- **Bivalvia** (class)
 - Range: Cambrian - Recent
- **Nautiloidea** (subclass)
 - Range: Cambrian - Recent
- **Stromatoporata** (class)
 - Range: Ordovician - Cretaceous
- **Eurypterida** (class)
 - Range: Cambrian - Permian
- **Graptolithina** (class)
 - Range: Cambrian - Carboniferous
- **Conodonta** (class)

- Range: Cambrian - Triassic
- **Insecta** (class)
 - Range: Silurian - Recent

The simplified workflow that can be used for evaluating the fossil occurrences of Articulata (the second fossil on the list; the fossil after Edrioasteroidea) is as follows:

```
using PaleobiologyDB, PaleobiologyDB.Taxonomy, DataFrames
using Makie, CairoMakie, GeoMakie, ColorSchemes

PaleobiologyDB.set_autocaching!(true);

occs_arti_raw = pbdb_occurrences(
  base_name = "Articulata",
  show = "full",
  vocab = "pbdb"
)

occs_arti = dropmissing(occs_arti_raw, [
  :genus,
  :early_interval,
  :max_ma,
  :min_ma
])

occs_arti = drop_unresolved_taxa(occs_arti, :genus)

occs_arti.midpoint_age = (occs_arti.max_ma .+ occs_arti.min_ma) ./ 2

occs_arti = sort(occs_arti, [:genus, :midpoint_age])

appearance_occs_arti = combine(
  groupby(occs_arti, :genus, sort = true),
  :midpoint_age => maximum => :first_appearance_datum,
  :midpoint_age => minimum => :last_appearance_datum
)

appearance_occs_arti = sort(appearance_occs_arti, :first_appearance_datum, rev=true)

range_through_chart_arti = DataFrame(
  genus = appearance_occs_arti.genus,
  first_appearance_datum = appearance_occs_arti.first_appearance_datum,
```

```

    last_appearance_datum = appearance_occs_arti.last_appearance_datum
)

fig = Figure(size = (800, 700))
ax = Axis(
    fig[1,1],
    xlabel = "Midpoint age (Ma) of Articulata genera",
    ylabel = "Genera",
    yticks = (1:nrow(appearance_occs_arti), appearance_occs_arti.genus),
    yticklabelsize = 4.5,
    title = "Ranges and diversity of Articulata genera",
    xreversed = true
)

for i in 1:nrow(appearance_occs_arti)
    x1 = appearance_occs_arti.first_appearance_datum[i]
    x2 = appearance_occs_arti.last_appearance_datum[i]
    lines!(ax, [x1, x2], [i, i], colorscale = :auto)
end

fig

```

Repeat this process for the other fossils of interests, and make sure to save each map for reference.

NOTE: In some of these charts, you might have to change the figure size to make it bigger (in resolution), which can be done by using the code `fig = Figure(size = (###, ###))` (the “###” can be changed to any number; e.g., 800x700). Alternatively, you might need to change the font size of the y-axis ticks through the attribute `yticklabelsize` (e.g., `yticklabelsize = 5`).

2 Part 2: Constructing a Diversity Curve

2.1 Preparation by counting diversity per geographical interval

We will now use `pbdb_occurrences` to retrieve data for our fossil occurrences (using *Edrioasteroidea* again as our example) and to proceed into the same steps for cleaning up our data. `PBDB.Taxonomy` is also used (as a module apart of “`PaleobiologyDB.jl`”) to drop any unresolved or unqualified taxa from our data. The code for these next few steps are as follows:

```

using PaleobiologyDB, DataFrames, CairoMakie, GeoMakie
PaleobiologyDB.set_autocaching!(true);

df_edri_diversity_raw = pbdb_occurrences(
    base_name = "Edrioasteroidea",
    show = "full",
    vocab = "pbdb"
)

df_edri_diversity = dropmissing(df_edri_diversity_raw, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])

df_edri_diversity = Taxonomy.drop_unqualified_taxa(df_edri_diversity, "genus")

```

We will then calculate our midpoint age (Ma) and use the split-apply-combine method to create a table of geographical intervals using a standard binning method. Specifically, we will be grouping, naming, and summarizing the representative age bounds of each of our fossil occurrences from oldest to youngest (using the `sort` function to align all ages in a reverse order). The code for these next few steps is below:

```

df_edri_diversity.midpoint_age = (df_edri_diversity.max_ma .+ df_edri_diversity.min_ma) ./ 2

interval_df_1 = combine(
    groupby(df_edri_diversity, :early_interval),
    :max_ma => maximum => :interval_max_ma,
    :min_ma => minimum => :interval_min_ma,
)

interval_df_1.interval_mid_ma = (
    interval_df_1.interval_max_ma .+ interval_df_1.interval_min_ma
) ./ 2

interval_df_1 = sort(interval_df_1, :interval_mid_ma, rev=true)

```

Next, we will be finding the first appearance and last appearance data for each of our genera in Edrioasteroidea by taking both the maximum and minimum ages per fossil occurrences. As such, this can be done by using the split-apply-combine method, as seen below:

```
genus_ranges_of_edri = combine(
  groupby(df_edri_diversity, :genus),
  :midpoint_age => maximum => :fad_ma,
  :midpoint_age => minimum => :lad_ma,
)
```

Now, we will create a diversity curve for all genera belonging to Edrioasteroidea using a `for...end` argument, which creates flow control using the index number `i` (which refers to both the rows and the y-axis). This serves to count one diversity number per geographical interval and to collect them inside a vector, as shown below:

```
diversity = Int[]

for interval_row in eachrow(interval_df_1)
  n = count(
    g -> g.fad_ma >= interval_row.interval_mid_ma >= g.lad_ma,
    eachrow(genus_ranges_of_edri)
  )
  push!(diversity, n)
end

interval_df_1.diversity = diversity
```

Make sure to perform these steps for the nineteen other fossils indicated above in the tutorial, which can help you prepare for the next step of this tutorial: Creating DataFrames for each of your data collections and making a high-quality visualization of your combined diversity curve using “Makie” family packages. A simplified workflow using *Articulata* as an example fossil of interest is demonstrated below. Notice how different variable names and components are used to distinguish each fossil from one another.

```
df_arti_diversity_raw = pbdb_occurrences(
  base_name = "Articulata",
  show = "full",
  vocab = "pbdb"
)

df_arti_diversity = dropmissing(df_arti_diversity_raw, [
  :genus,
  :early_interval,
  :max_ma,
  :min_ma
])
```

```

])

df_arti_diversity = Taxonomy.drop_unqualified_taxa(df_arti_diversity, "genus")

df_arti_diversity.midpoint_age = (df_arti_diversity.max_ma .+ df_arti_diversity.min_ma) ./ 2

interval_df_2 = combine(
  groupby(df_arti_diversity, :early_interval),
  :max_ma => maximum => :interval_max_ma,
  :min_ma => minimum => :interval_min_ma,
)

interval_df_2.interval_mid_ma = (
  interval_df_2.interval_max_ma .+ interval_df_2.interval_min_ma
) ./ 2

interval_df_2 = sort(interval_df_2, :interval_mid_ma, rev=true)

genus_ranges_of_arti = combine(
  groupby(df_arti_diversity, :genus),
  :midpoint_age => maximum => :fad_ma,
  :midpoint_age => minimum => :lad_ma,
)

diversity = Int[]

for interval_row in eachrow(interval_df_2)
  n = count(
    g -> g.fad_ma >= interval_row.interval_mid_ma >= g.lad_ma,
    eachrow(genus_ranges_of_arti)
  )
  push!(diversity, n)
end

interval_df_2.diversity = diversity

```

The output shows a vector of different diversity numbers being counted per geographic interval in the realm of geospatial time. This is because the `for... end` argument serves as a function in collecting these numbers (represented by first appearance and last appearance data) into a combined vector with the `Int64` type. This is true for every time you perform this certain function (or code) for each of your diversity loops.

2.2 Creating DataFrames for each fossil

Before we move on to creating our diversity curves, we will need to create DataFrames for each of our twenty diversity loops/interval datum. This is to ensure that everything is getting graphed directly onto the axes without running the risk for errors. These include `DimensionMismatch` errors, which usually occur when certain arrays (e.g., our data) cannot be broadcast due to a mismatch in size, resolution, or area length.

For each DataFrame, we will include data for both the diversity and geographical intervals for each of our fossils of interest. We will be using *Edrioasteroidea* as an example for our demonstration. In this case, we will be manually creating the DataFrame by loading “DataFrames.jl” into the environment, then by creating our variable name. Each variable name (for each of our fossils) should be distinct from one another to avoid confusion. For each variable, two new columns will be created: one for diversity and another for geographical intervals, as seen below.

```
range_through_1 = DataFrame(  
    edrioasteroidea_diversity = interval_df_1.diversity,  
    interval_mid_ma_edri = interval_df_1.interval_mid_ma  
)
```

Using the workflow established above, create DataFrames for the remaining nineteen fossils of interest, then proceed into the next section.

2.3 Plotting a diversity curve

For this particular graph, since there are so many fossils of interest (per this tutorial), we will be splitting the graph into four separate axes. This can be done by using the ‘Axis’ function via the “Makie” family of packages and by changing the numbers in the `fig` attribute (e.g., `fig[1,1]` to indicate an axis to the left). We will need 20 axes total (from `ax1` to `ax20`), so five axes will be put into each new figure (e.g., `fig`) for a total of four big graphs. `hidedecorations!` is used to hide the descriptions for both x- and y-axes, as well as certain attributes pertaining to the graph, which is crucial for these types of graphs, since we don’t want to overwhelm our visualization with too much cramped information. As seen in the code below, twenty different `lines!` arguments and twenty different `scatter!` arguments are to be used to plot each DataFrame (as established in the previous section) separately, and by each different color.

For reference, the code below mainly consists of commands previously established throughout the tutorial. Additionally, many components were copied-and-pasted manually by keyboard shortcuts to show repetition of the functions needed for the graph. The full, complex code for this process is below, as well as the final resulting visualization for one of the four diversity curves needed (for `ax1` to `ax5`):

```

using Makie, CairoMakie, ColorSchemes
fig = Figure(size = (1200, 900), fontsize = 11)
ax1 = Axis(
    fig[1, 1],
    xlabel = "Age (Ma)",
    ylabel = "Number of genera",
    title = "Genus diversity of several fossils through time",
    xticks = (interval_df_1.interval_mid_ma, interval_df_1.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 6.5,
    xreversed = true,
)
ax2 = Axis(
    fig[1,1],
    xticks = (interval_df_2.interval_mid_ma, interval_df_2.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 6.5,
    xreversed = true,
)
ax3 = Axis(
    fig[1,1],
    xticks = (interval_df_3.interval_mid_ma, interval_df_3.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 6.5,
    xreversed = true,
)
ax4 = Axis(
    fig[1,1],
    xticks = (interval_df_4.interval_mid_ma, interval_df_4.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 6.5,
    xreversed = true,
)
ax5 = Axis(
    fig[1,1],
    xticks = (interval_df_5.interval_mid_ma, interval_df_5.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 6.5,
    xreversed = true,
)
hidedecorations!(ax2)
hidedecorations!(ax3)

```

```

hidedecorations!(ax4)
hidedecorations!(ax5)

lines!(ax1, interval_df_1.interval_mid_ma,
        interval_df_1.diversity, linewidth = 2, color = :red4)
scatter!(ax1, interval_df_1.interval_mid_ma,
          interval_df_1.diversity, markersize = 6, color = :red4, label = "Edrioasteroidea")

lines!(ax2, interval_df_2.interval_mid_ma,
        interval_df_2.diversity, linewidth = 2, color = :goldenrod)
scatter!(ax2, interval_df_2.interval_mid_ma,
          interval_df_2.diversity, markersize = 6, color = :goldenrod, label = "Articulata")

lines!(ax3, interval_df_3.interval_mid_ma,
        interval_df_3.diversity, linewidth = 2, color = :dodgerblue)
scatter!(ax3, interval_df_3.interval_mid_ma,
          interval_df_3.diversity, markersize = 6, color = :dodgerblue, label = "Ammonoidea")

lines!(ax4, interval_df_4.interval_mid_ma,
        interval_df_4.diversity, linewidth = 2, color = :palevioletred)
scatter!(ax4, interval_df_4.interval_mid_ma,
          interval_df_4.diversity, markersize = 6, color = :palevioletred, label = "Rugosa")

lines!(ax5, interval_df_5.interval_mid_ma,
        interval_df_5.diversity, linewidth = 2, color = :darkorchid3)
scatter!(ax5, interval_df_5.interval_mid_ma,
          interval_df_5.diversity, markersize = 6, color = :darkorchid3, label = "Tabulata")

axislegend(ax1, position = (0.01, 0.975), framevisible = true, labelsz = 11)
axislegend(ax2, position = (0.01, 0.925), framevisible = true, labelsz = 11)
axislegend(ax3, position = (0.01, 0.875), framevisible = true, labelsz = 11)
axislegend(ax4, position = (0.01, 0.825), framevisible = true, labelsz = 11)
axislegend(ax5, position = (0.01, 0.775), framevisible = true, labelsz = 11)

fig

```

Ensure that you repeat this workflow for your three other diversity curves, including one for ax6 to ax10, another for ax11 to ax15, and one last curve for ax16 to ax20. Make sure to include legends into your graphs by using `axislegend` and any attributes that might be useful, like `position`, `labelsize`, and `framevisible`.

2.4 Why are we doing all of this?

The main goal of making these diversity curves is to measure changes in biodiversity between different geospatial intervals and events in geological history for different taxonomic ranks. Here, we will be providing a simplified workflow of creating a diversity curve using just one group of organisms. This time, we will be using Edrioasteroidea as our example again, along with another attribute known as `axislegend()`. This gives us a legend of what point are there on our graph, as demonstrated below:

```
using Makie, CairoMakie, ColorSchemes

fig = Figure(fontsize = 11)
ax = Axis(
    fig[1, 1],
    xlabel = "Age (Ma)",
    ylabel = "Number of genera",
    title = "Genus diversity of Edrioasteroidea fossils through time",
    xticks = (interval_df_1.interval_mid_ma, interval_df_1.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 6.5,
    xreversed = true,
)
lines!(ax, interval_df_1.interval_mid_ma,
        interval_df_1.diversity, linewidth = 3, color = :green)
scatter!(ax, interval_df_1.interval_mid_ma,
        interval_df_1.diversity, markersize = 9, color = :green, label = "Edrioasteroidea")

axislegend(position = :lt)

fig
```

3 Part 3: Answering Activity Questions

Based on the range through chart and diversity curves that were generated for our twenty fossils, you may answer the following questions, which are also listed in the original activity/tutorial PDF. It might be best to pull up your original activity file and to follow through it for this section.

Question 1: When did the largest extinction event occur, and how many of the twenty organisms went extinct by then? [Bulinski, 2009]

Question 2: What caused this extinction? [Extension of Question 1; Bulinski, 2009]

Question 3: Which major extinction caused ammonoids (subclass Ammonoidea) to go extinct? What caused this extinction event to occur? [Bulinski, 2009]

Question 4: During what geological period do the most taxa originate, and what is the name of this origination event? [Bulinski, 2009]

Question 5: During what geological period to the second largest number of taxa originate. What is the name of this origination event? [Bulinski, 2009]

Question 6: Notice when the three orders of coral (Rugosa, Tabulata, and Scleractinia) originate and go extinct. Now, explain, in a few sentences, what the fossil record of corals looks like as of today. [Bulinski, 2009]

Question 7: How many of the twenty taxa are extant (still in existence) today? [Bulinski, 2009]

Question 8: Explain three similarities and three differences between the diversity curve that you generated with Julia and the Sepkoski Curve. Why are these curves different from one another? [Bulinski, 2009]

Question 9: If you constructed a range through chart of fossil genera, would you expect their ranges in the fossil record to be longer or shorter than those of classes and/or orders? Why? [Bulinski, 2009]

Question 10: Why do you not find many fossils in the rock record before the Cambrian period? [Bulinski, 2009]

For simplicity's sake, Questions 1 to 6 can be answered by using your diversity curves (from **Part 2**) and range through charts (from **Part 1**). Questions 7 to 10 can be answered by using both the activity material and any additional online materials (e.g., the PBDB and its Navigator tool). They operate as short answer questions, or in some cases as hypothetical questions (e.g., Questions 8 and 9).

4 References (for Original Activity)

Bulinski, K. (2009, June 4). Using Range Through Charts: Constructing a Diversity Curve. Cutting Edge. <https://serc.carleton.edu/NAGTWorkshops/paleo/activities/33101.html>

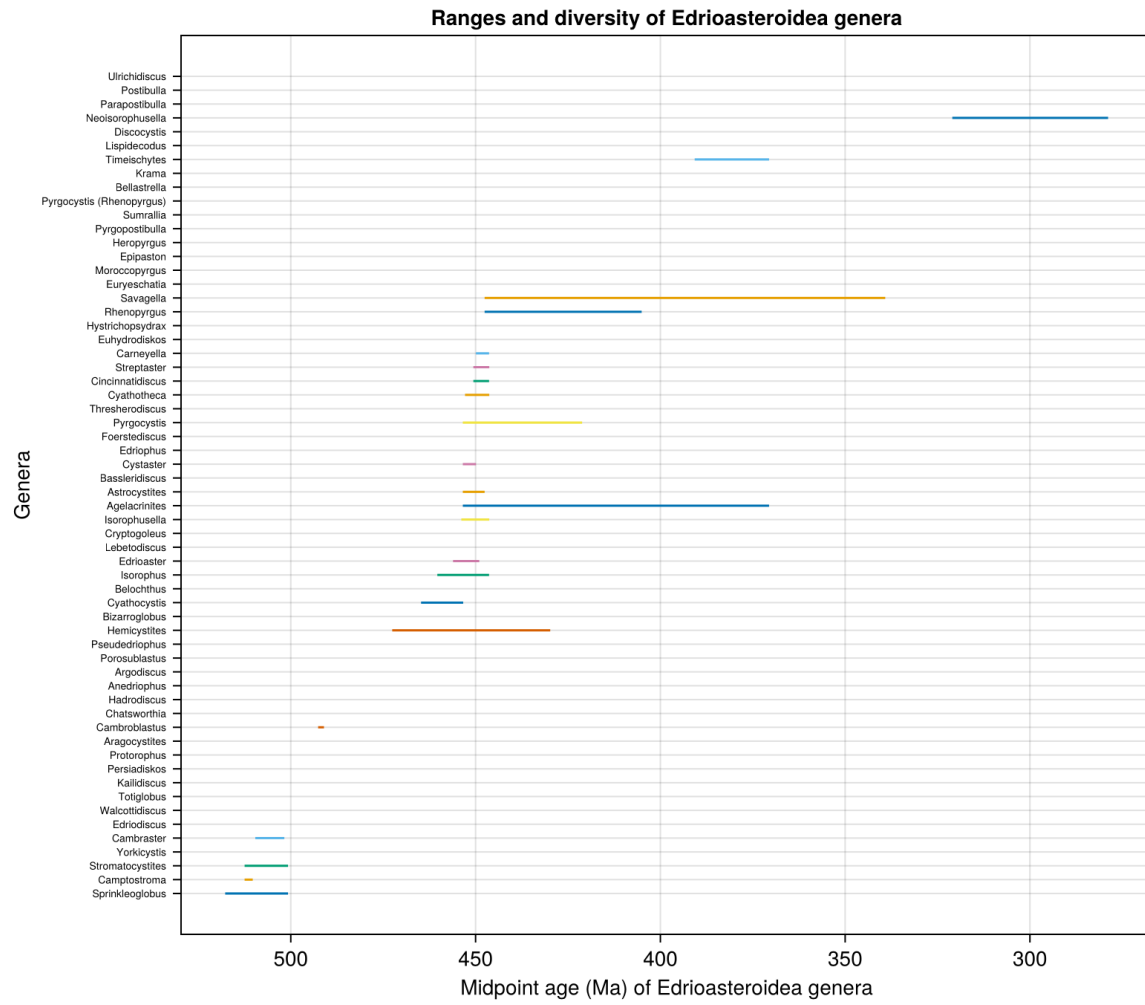


Figure 1: Ranges and diversity of Edrioasteroidea genera through geological time

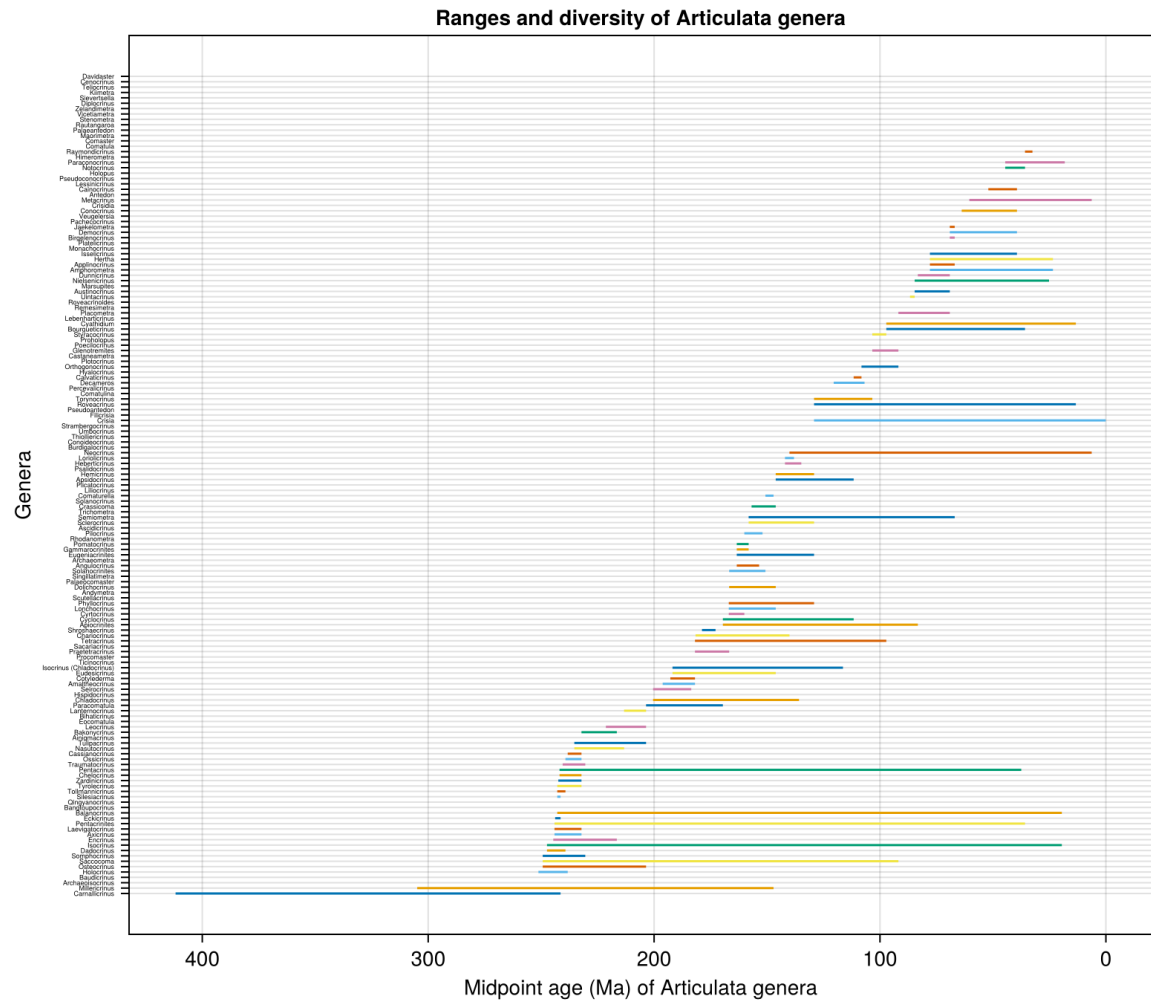


Figure 2: Ranges and diversity of Articulata genera through geological time

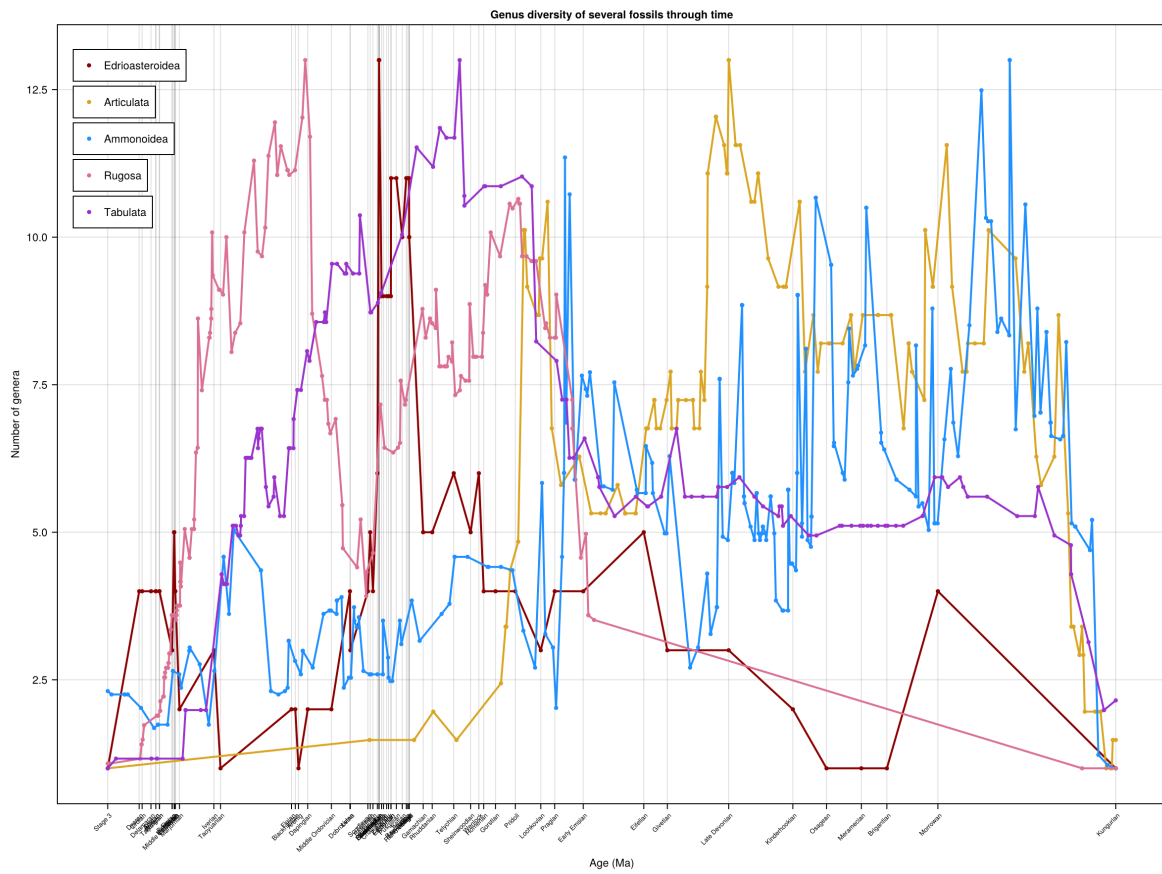


Figure 3: One of four diversity curves for Edrioasteroidea, Articulata, Ammonoidea, Rugosa, and Tabulata

